

Jira-lite Project Complexity and Cost Report

Generated: 2026-02-09 Repository: /mnt/e/jira-lite

1) Executive Summary

Jira-lite is a **medium-high complexity full-stack product** with:

- Multi-space tenancy and role-based access control
- Stateful AI assistant orchestration and tooling
- Document workflows (Workbench) and decomposition workflows (Structure Studio)
- Real-time updates, planning/capacity management, and CSV import/export pipelines

It is not a simple CRUD app. The architecture includes multiple domains (delivery execution, capacity planning, admin/governance, AI-assisted workflows) and substantial integration behavior across backend, frontend, and tests.

2) Measured Size (LOC and File Count)

Method:

- Counts below are from `wc -l` over source files in this repo
- Includes blank/comment lines (physical LOC), which is standard for scoping

2.1 Major component totals

Component	Files	LOC
Backend Python (src/main/backend)	46	20,223
Frontend UI (src/main/ui JS/HTML/CSS)	16	13,004
Tests (src/main/test)	120	5,637
Prompts (prompts)	15	607
Templates (templates)	7	254
Scripts (scripts.py/.sh)	5	396
Docs (docs.md)	1	466
Total (above scope)	210	40,587

2.2 Backend breakdown

Backend subsystem	Files	LOC	Share of backend
API routes (app/routes)	15	9,275	45.9%
AI subsystem (app/ai)	8	7,713	38.1%
Services (app/services)	8	1,143	5.7%
Schemas (app/schemas)	1	804	4.0%
Models (app/models)	1	607	3.0%
Core infra/auth/db/utils/root	13	681	3.4%
Backend total	46	20,223	100%

Largest backend files:

- `src/main/backend/app/ai/tools.py` (3,715 LOC)
- `src/main/backend/app/ai/orchestrator.py` (3,686 LOC)
- `src/main/backend/app/routes/workbench.py` (2,351 LOC)
- `src/main/backend/app/routes/ai.py` (1,456 LOC)

2.3 Frontend breakdown

Frontend area	Files	LOC	Share of frontend
Main client controller (src/main/ui/js/app.js)	1	7,938	61.0%
Styling (src/main/ui/styles.css)	1	2,665	20.5%
HTML shell (src/main/ui/index.html)	1	1,067	8.2%
Route modules (src/main/ui/js/routes)	12	1,334	10.3%
Frontend total	15*	13,004	100%

* `src/main/ui` has 16 total files; this table scopes the primary runtime UI files.

2.4 Test scale

- Test files: **33** (`test_*.py`)
- Test functions: **84** (`def test_...`)
- Test LOC: **5,637**

Largest test files:

- `src/main/test/test_ai_orchestrator.py` (522 LOC)
- `src/main/test/test_structure_studio.py` (477 LOC)
- `src/main/test/test_subcomponents.py` (364 LOC)
- `src/main/test/test_workbench.py` (359 LOC)

3) Architectural Complexity Indicators

3.1 API and domain surface

- FastAPI route handlers: **132** (`@router.get/post/patch/put/delete`)
- Core route-heavy modules:
 - `ai.py` (27 handlers)
 - `workbench.py` (19 handlers)
 - `users.py`, `subcomponents.py`, `auth.py` (10 each)
- SQLAlchemy model classes: **29** total classes in model file (includes base class; roughly 28 domain entities)

3.2 AI platform depth

- AI tool catalog entries: **33** tools
- Dedicated orchestration engine + JSON/tool-call control loop
- Usage-guide retrieval path (`explain_app_usage`) + contextual routing
- AI write-approval flow and guarded persistence logic

3.3 Product complexity drivers

High-complexity drivers in this codebase:

- Space-scoped tenancy and permission enforcement
- Role governance constraints (global admin + space admin safety rules)
- Document lifecycle state transitions and validation rules
- Hybrid planning model (resource allocations/windows/KPIs)
- Import/export workflows with partial-success error semantics
- Real-time sync + cache invalidation + route-based UI module loading

4) What It Would Take to Build (Pre-AI Coding Agents)

Assumptions for this estimate:

- Build from scratch to comparable functionality and reliability
- US-based engineering org, normal SDLC (discovery, build, QA, UAT, hardening)
- Includes backend, frontend, auth, multi-space RBAC, tests, and AI workflow integration
- Excludes major custom ML model training (uses provider APIs)

4.1 Typical delivery team

- 1 Tech Lead / Architect
- 2 Backend Engineers
- 2 Frontend Engineers
- 1 QA Engineer (automation + exploratory)
- 0.5 DevOps/SRE
- 0.5 Product + 0.25 UX/Design (shared)

Effective core team: ~6–7 FTE over most of the project.

4.2 Effort by workstream (pre-AI baseline)

Workstream	Person-weeks
Discovery, architecture, domain modeling	20–30
Auth + space/role governance + security hardening	25–40
Core delivery domains (projects/solutions/subcomponents + import/export)	45–65
Planning, dashboard, calendar/kanban, admin screens	35–55
AI assistant orchestration + tooling + approval workflows	40–65
Document Studio + Structure Studio workflows	35–55
QA automation, UAT, stabilization, release hardening	25–45
Total	225–355 person-weeks

4.3 Calendar-time estimate (pre-AI)

- **Lean but senior team (6–7 FTE):** ~8–11 months
- **Typical enterprise sequencing:** ~10–14 months
- **High-governance program (heavy compliance/review):** ~12–16 months

5) Cost Estimate (Pre-AI Coding Agents)

5.1 Hour conversion

- 225–355 person-weeks × 40 hours = **9,000–14,200 hours**

5.2 Cost ranges (US market)

Delivery model	Blended rate	Estimated cost
Internal tech department (fully loaded)	\$95–\$140/hr	\$855k–\$1.99M

External product engineering vendor	\$140–\$220/hr	\$1.26M–\$3.12M
Premium consultancy / high-compliance context	\$220–\$300/hr	\$1.98M–\$4.26M

6) Why This Is Not a “Simple App” Cost

The main cost multipliers are:

- Multi-tenant security and role/space governance correctness
- AI orchestration reliability (tool routing, approval state, error handling)
- Complex workflow/state transitions (Workbench + Structure Studio)
- Large UI state surface and integration-heavy behavior
- Regression-risk control via a non-trivial automated test suite

7) Confidence and Caveats

Confidence: **Medium** for budgetary planning.

Biggest variability factors:

- Non-functional requirements (security/compliance/audit depth)
- Team skill and domain familiarity
- Scope strictness (MVP vs parity with current repo behavior)
- Environment and deployment requirements (SRE/compliance burden)

If you want, I can generate a second version of this report with:

1. A **strict MVP estimate** (must-have only)
2. A **parity estimate** (closest to current repo behavior)
3. A **phase-by-phase staffing plan** by month

--	--	--